

MileStone 1 Sample Questions

SQL:

Easy: Customer with the Highest Value Order

Database Description

The retail_store database is structured to manage and analyze key transactional and operational data within a retail environment. It captures detailed information about customers, product inventory, orders, and item-level purchases. This schema supports querying customer purchase behavior, product sales performance, and order histories, enabling insights into customer trends, product demand, and overall business operations

Schema Specification

Table: Customers

Stores customer profile and contact information.

Column Name	Data Type	Constraints	Description
customer_id	INT	PRIMARY KEY	Unique ID for each customer
name	VARCHAR(100)	NOT NULL	Customer full name
email	VARCHAR(100)	UNIQUE NOT NULL	Email address of the customer
phone	VARCHAR(20)		Contact number

Table: Products

Stores product catalog and pricing information.

Column Name	Data Type	Constraints	Description
product_id	INT	PRIMARY KEY	Unique product identifier
name	VARCHAR(100)	NOT NULL	Name of the product
category	VARCHAR(50)		Category to which product belongs
price	DECIMAL(10,2)	NOT NULL	Price per unit

■ Table: Orders

Captures order-level metadata including customer and date.

Column Name	Data Type	Constraints	Description
order_id	INT	PRIMARY KEY	Unique order number
customer_id	INT	FOREIGN KEY → Customers(customer_id)	Customer who placed the order
order_date	DATE	NOT NULL	Date the order was placed
total_amount	DECIMAL(10,2)		Total bill amount for the order

■ Table: Order_Items

Represents items associated with each order, along with quantity and pricing.

Column Name	Data Type	Constraints	Description
order_id	INT	FOREIGN KEY → Orders(order_id)	Order to which item belongs
product_id	INT	FOREIGN KEY → Products(product_id)	Product purchased in the order
quantity	INT	NOT NULL	Number of units ordered
unit_price	DECIMAL(10,2)	NOT NULL	Price per unit at the time of order
PRIMARY KEY	Composite	PRIMARY KEY (order_id, product_id)	Ensures one entry per product/order

✔ Test Data

Customers

customer_id	name	email	phone
201	John Doe	john@example.com	1234567890
202	Jane Smith	jane@example.com	2345678901
203	Emily Davis	emily@example.com	3456789012
204	Mark Wilson	mark@example.com	4567890123

Products

product_id	name	category	price
301	Laptop	Electronics	1000.00
302	Smartphone	Electronics	600.00
303	Office Chair	Furniture	150.00
304	Notebook Set	Stationery	10.00

Orders

order_id	customer_id	order_date	total_amount
401	201	2023-08-01	1010.00
402	202	2023-08-03	1600.00
403	203	2023-08-04	20.00
404	204	2023-08-05	750.00

Order_Items

order_id	product_id	quantity	unit_price
401	301	1	1000.00
401	304	1	10.00
402	301	1	1000.00
402	302	1	600.00
403	304	2	10.00
404	303	5	150.00
404	304	1	10.00
402	303	1	150.00
403	302	1	600.00
401	303	2	150.00
403	301	1	1000.00
404	302	1	600.00

Problem Statement

Management wants to know which customer placed the single **highest-value order**. You need to find and return the customer's name along with the order_id and total_amount of that order.

Query to be Implemented

Write a query that uses a **single-row subquery** to find:

order_id

customer_name

total_amount

for the order with the **maximum total_amount** in the Orders table.

Expected Oupput

order_id	customer_name	total_amount
402	Jane Smith	1600.00

 SQL

```
SELECT
    o.order_id,
    c.name AS customer_name,
    o.total_amount
FROM Orders o
JOIN Customers c
    ON o.customer_id = c.customer_id
WHERE o.total_amount = (
    SELECT MAX(total_amount)
    FROM Orders
);
```

SQL Easy: Student Enrollments with Grades and Department Names

Database Description

The university_core database is designed to manage and organize essential academic information within a university setting. It maintains comprehensive records of departments, students, courses, and enrollments. The system facilitates tracking of student-course registrations, departmental offerings, and student grades, supporting academic administration and reporting needs.

Schema Specification

Table: Departments

Stores all academic departments within the university.

Column Name	Data Type	Constraints	Description
department_id	INT	PRIMARY KEY	Unique identifier for department
department_name	VARCHAR(50)	NOT NULL	Name of the department

Table: Students

Stores student information along with their department.

Column Name	Data Type	Constraints	Description
student_id	INT	PRIMARY KEY	Unique student ID
name	VARCHAR(100)	NOT NULL	Full name of the student
age	INT		Age of the student
department_id	INT	FOREIGN KEY → Departments(department_id)	Student's home department

Table: Courses

Stores the list of courses offered by departments.

Column Name	Data Type	Constraints	Description
course_id	VARCHAR(10)	PRIMARY KEY	Unique course identifier
course_name	VARCHAR(100)	NOT NULL	Name of the course
department_id	INT	FOREIGN KEY → Departments(department_id)	Department offering the course

📊 **Table: Enrollments**

Stores student enrollments in courses along with their grades.

Column Name	Data Type	Constraints	Description
student_id	INT	FOREIGN KEY → Students(student_id)	Enrolled student ID
course_id	VARCHAR(10)	FOREIGN KEY → Courses(course_id)	Course enrolled in
grade	VARCHAR(5)		Grade achieved in the course
PRIMARY KEY	Composite	PRIMARY KEY (student_id, course_id)	Composite key to prevent duplicates

✔ **Test Data**

Departments

department_id	department_name
1	Computer Science
2	Mathematics
3	Physics
4	Literature

Students

student_id	name	age	department_id
101	Alice Smith	20	1
102	Bob Johnson	22	2
103	Clara Evans	21	3
104	David Kumar	23	1

Courses

course_id	course_name	department_id
C101	Algorithms	1
C102	Linear Algebra	2
C103	Quantum Mechanics	3
C104	Shakespearean Drama	4

Enrollments

student_id	course_id	grade
101	C101	A
101	C102	B+
101	C104	A-
102	C102	B
102	C103	A
103	C103	B-
103	C104	A
104	C101	A+
104	C103	B
104	C102	C+
104	C104	B
101	C103	B+

Problem Statement

Generate a report listing every student along with the courses they are enrolled in, the grades they have received, and the departments offering those courses. This report provides a complete academic view of student performance along with academic affiliations.

Query to be Implemented

Write a single SQL query to fetch the following details for each student-course enrollment:

- student_id
- student_name
- course_id

- course_name
- grade
- department_name (offering the course)

Expected Output

Query Result					
student_id	student_name	course_id	course_name	grade	department_name
101	Alice Smith	C101	Algorithms	A	Computer Science
101	Alice Smith	C102	Linear Algebra	B+	Mathematics
101	Alice Smith	C103	Quantum Mechanics	B+	Physics
101	Alice Smith	C104	Shakespearean Drama	A-	Literature
102	Bob Johnson	C102	Linear Algebra	B	Mathematics
102	Bob Johnson	C103	Quantum Mechanics	A	Physics
103	Clara Evans	C103	Quantum Mechanics	B-	Physics
103	Clara Evans	C104	Shakespearean Drama	A	Literature
104	David Kumar	C101	Algorithms	A+	Computer Science
104	David Kumar	C102	Linear Algebra	C+	Mathematics
104	David Kumar	C103	Quantum Mechanics	B	Physics
104	David Kumar	C104	Shakespearean Drama	B	Literature

 SQL

```
SELECT
  s.student_id,
  s.name AS student_name,
  c.course_id,
  c.course_name,
  e.grade,
  d.department_name
FROM Students s
JOIN Enrollments e
  ON s.student_id = e.student_id
JOIN Courses c
  ON e.course_id = c.course_id
JOIN Departments d
  ON c.department_id = d.department_id
ORDER BY s.student_id, c.course_id;
```

SQL Easy : Calculate the average assignment score per course

Calculate the average assignment score per course

Table 1 - Students

Table 2 - Instructor

Table 3 - Courses

Table 4 - Enrollment

Table 5 - Assignments

Table 6 - Submissions

Join suitable tables to get course name & average score.

Sample Schema

Student Schema

	student_id	student_name	email	registration_date
	1	Alice Johnson	alice@example.com	2024-01-05
	2	Bob Smith	bob@example.com	2023-12-15
	3	Charlie Brown	charlie@example.com	2023-11-20
	4	David Williams	david@example.com	2023-10-10
	5	Emma Davis	emma@example.com	2024-02-01
	6	Frank Thomas	frank@example.com	2024-03-05
	7	Grace Miller	grace@example.com	2024-02-18

Instructor Schema

	instructor_id	instructor_name	email	department
	1	Dr. John Doe	john.doe@university.com	Computer Science
	2	Dr. Sarah Lee	sarah.lee@university.com	Mathematics
	3	Dr. Michael Johnson	michael.j@university.com	Physics

Courses Schema

	course_id	course_name	instructor_id	start_date	end_date
	1	Database Management	1	2024-01-10	2024-06-30
	2	Machine Learning	2	2024-02-15	2024-07-30
	3	Data Structures	1	2023-09-01	2024-04-01
	4	Quantum Mechanics	3	2024-03-01	2024-08-30
	5	Calculus II	2	2024-04-05	2024-09-15

Enrollment Schema

	enrollment_id	student_id	course_id	enrollment_date
	1	1	1	2024-01-12
	2	2	1	2024-01-14
	3	3	2	2024-02-20
	4	4	2	2024-02-22
	5	5	3	2023-09-10
	6	6	3	2023-10-05
	7	7	4	2024-03-05
	8	1	5	2024-04-10
	9	2	4	2024-03-12
	10	3	5	2024-04-15

Assignments

	assignment_id	course_id	assignment_title	due_date	max_score
	1	1	SQL Basics Assignment	2024-02-10	100
	2	1	Advanced SQL Queries	2024-03-15	100
	3	2	Linear Regression Project	2024-03-01	100
	4	2	Neural Networks Report	2024-04-05	100
	5	3	Sorting Algorithms	2023-10-10	100
	6	3	Graph Theory Assignment	2023-12-15	100
	7	4	Schrodinger's Equation Analysis	2024-04-01	100
	8	5	Integration Techniques	2024-05-10	100

Submissions

	submission_id	student_id	assignment_id	submission_date	score
	1	1	1	2024-02-08 00:00:00	85
	2	2	1	2024-02-09 00:00:00	92
	3	3	2	2024-03-14 00:00:00	78
	4	4	2	2024-03-15 00:00:00	95
	5	5	3	2024-02-28 00:00:00	89
	6	6	4	2024-04-03 00:00:00	76
	7	1	5	2023-10-08 00:00:00	80
	8	1	6	2023-12-14 00:00:00	88
	9	2	7	2024-03-30 00:00:00	94
	10	3	8	2024-05-08 00:00:00	90

Expected Output

	course_name	avg_score
	Quantum Mechanics	94.0000
	Calculus II	90.0000
	Database Management	87.5000
	Data Structures	84.0000
	Machine Learning	82.5000

`</>` SQL

```
SELECT
    c.course_name,
    AVG(s.score) AS avg_score
FROM Courses c
JOIN Assignments a
    ON c.course_id = a.course_id
JOIN Submissions s
    ON a.assignment_id = s.assignment_id
GROUP BY c.course_name
ORDER BY avg_score DESC;
```

Expected Output

course_name	avg_score
Quantum Mechanics	94.0000
Calculus II	90.0000
Database Management	87.5000
Data Structures	84.0000
Machine Learning	82.5000

SQL MEDIUM: Highest-Revenue Doctor per Department

Problem Statement

A hospital administration system tracks appointments and billing for doctors across multiple departments.

Each doctor belongs to a specific department, and each completed appointment generates a bill recorded in the **Bills** table.

You are asked to generate a report that shows the **top-earning doctors within each department**, based on the total billed amount from all their appointments.

For every doctor, compute:

- The department name
- The doctor name
- The total revenue generated (sum of all their bill amounts)
- The rank of that doctor within the department (highest revenue = rank 1)

Use RANK() (not ROW_NUMBER), so that if two doctors have the same total revenue, **they receive the same rank**.

Required output columns

- department_name
- doctor_name
- total_revenue
- revenue_rank

Schema Details

Database: HOSPITAL_DB

1) Departments

Column Name	Data Type	Description
dept_id	INT (PK)	Department ID
department_name	VARCHAR(100)	Name of department

2) Doctors

Column Name	Data Type	Description
doctor_id	INT (PK)	Unique doctor ID
doctor_name	VARCHAR(100)	Doctor's full name
dept_id	INT (FK)	Department doctor belongs to

3) Appointments

Column Name	Data Type	Description
appointment_id	INT (PK)	Appointment ID
doctor_id	INT (FK)	Doctor who handled the appointment
appointment_date	DATE	Date of appointment

4) Bills

Column Name	Data Type	Description
bill_id	INT (PK)	Unique bill ID
appointment_id	INT (FK)	Appointment for which bill was generated
amount	DECIMAL(10,2)	Bill amount

Sample Data Section

Departments

dept_id	department_name
1	Cardiology
2	Neurology

Doctors

doctor_id	doctor_name	dept_id
1	Dr. Smith	1
2	Dr. Anand	1
3	Dr. Lee	1
4	Dr. Patel	2
5	Dr. Ray	2

Appointments

appointment_id	doctor_id	appointment_date
101	1	2025-11-17
102	1	2025-11-15
103	2	2025-11-16
104	2	2025-11-14
105	3	2025-11-14
106	4	2025-11-17
107	4	2025-11-16
108	5	2025-11-17

Bills

bill_id	appointment_id	amount
201	101	800.00
202	102	700.00
203	103	600.00
204	104	600.00
205	105	1200.00
206	106	1000.00
207	107	1000.00
208	108	900.00

Expected Output (Exact)

For the given test dataset, the expected output is:

department_name	doctor_name	total_revenue	revenue_rank
Cardiology	Dr. Smith	1500.00	1
Cardiology	Dr. Anand	1200.00	2
Cardiology	Dr. Lee	1200.00	2
Neurology	Dr. Patel	2000.00	1
Neurology	Dr. Ray	900.00	2

Note: Because RANK() is used, doctors with equal total revenue share the same rank (e.g., **Dr. Anand** and **Dr. Lee** both ranked 2 in Cardiology).

Expected output

department_name	doctor_name	total_revenue	revenue_rank
Cardiology	Dr. Smith	1500.00	1
Cardiology	Dr. Anand	1200.00	2
Cardiology	Dr. Lee	1200.00	2
Neurology	Dr. Patel	2000.00	1
Neurology	Dr. Ray	900.00	2

`</>` SQL

```
SELECT
    d.department_name,
    dr.doctor_name,
    SUM(b.amount) AS total_revenue,
    RANK() OVER (
        PARTITION BY d.department_name
        ORDER BY SUM(b.amount) DESC
    ) AS revenue_rank
FROM Departments d
JOIN Doctors dr
    ON d.dept_id = dr.dept_id
JOIN Appointments a
    ON dr.doctor_id = a.doctor_id
JOIN Bills b
    ON a.appointment_id = b.appointment_id
GROUP BY
    d.department_name,
    dr.doctor_name;
```

SQL Medium: Students Ranked by Course Enrollments

✓ Database Description

The **University Core** database is designed to manage and organize essential academic information within a university setting. It maintains comprehensive records of departments, students, courses, and enrollments. The system facilitates the tracking of student-course registrations, departmental offerings, and the grading history of students, ensuring efficient academic administration and reporting.

✓ Schema Specification

📁 Departments

Stores all academic departments within the university.

Column Name	Data Type	Constraints	Description
department_id	INT	PRIMARY KEY	Unique identifier for department
department_name	VARCHAR(50)	NOT NULL	Name of the department

📁 Students

Stores student information along with their department.

Column Name	Data Type	Constraints	Description
student_id	INT	PRIMARY KEY	Unique student ID
name	VARCHAR(100)	NOT NULL	Full name of the student
age	INT		Age of the student
department_id	INT	FOREIGN KEY → Departments(department_id)	Student's home department

📁 Courses

Stores the list of courses offered by departments.

Column Name	Data Type	Constraints	Description
course_id	VARCHAR(10)	PRIMARY KEY	Unique course identifier
course_name	VARCHAR(100)	NOT NULL	Name of the course
department_id	INT	FOREIGN KEY → Departments(department_id)	Department offering the course

Enrollments

Stores student enrollments in courses along with their grades.

Column Name	Data Type	Constraints	Description
student_id	INT	FOREIGN KEY → Students(student_id)	Enrolled student ID
course_id	VARCHAR(10)	FOREIGN KEY → Courses(course_id)	Course enrolled in
grade	VARCHAR(5)		Grade achieved in the course
PRIMARY KEY	Composite	PRIMARY KEY (student_id, course_id)	Composite key to prevent duplicates

Sample Data

Departments

department_id	department_name
1	Computer Science
2	Mathematics
3	Physics
4	Literature

Students

student_id	name	age	department_id
101	Alice Smith	20	1
102	Bob Johnson	22	2
103	Clara Evans	21	3
104	David Kumar	23	1

Courses

course_id	course_name	department_id
C101	Algorithms	1
C102	Linear Algebra	2
C103	Quantum Mechanics	3
C104	Shakespearean Drama	4

📊 Enrollments

student_id	course_id	grade
101	C101	A
101	C102	B+
101	C104	A-
102	C102	B
102	C103	A
103	C103	B-
103	C104	A
104	C101	A+
104	C103	B
104	C102	C+
104	C104	B
101	C103	B+

✅ Problem Statement

Write a query to assign a **RANK** and a **DENSE_RANK** to each student based on the number of courses they are enrolled in (in descending order). If multiple students are enrolled in the same number of courses, they should receive the same rank.

Return the output with the following columns:

- student_id
- name
- course_count (total number of enrollments per student)
- rank_by_enrollments (using RANK())
- dense_rank_by_enrollments (using DENSE_RANK())

Order the final result by rank_by_enrollments.

✓ Expected Output

student_id	name	course_count	rank_by_enrollments	dense_rank_by_enrollments
101	Alice Smith	4	1	1
104	David Kumar	4	1	1
102	Bob Johnson	2	3	2
103	Clara Evans	2	3	2

SQL

```
SELECT
  s.student_id,
  s.name,
  COUNT(e.course_id) AS course_count,
  RANK() OVER (
    ORDER BY COUNT(e.course_id) DESC
  ) AS rank_by_enrollments,
  DENSE_RANK() OVER (
    ORDER BY COUNT(e.course_id) DESC
  ) AS dense_rank_by_enrollments
FROM Students s
JOIN Enrollments e
  ON s.student_id = e.student_id
GROUP BY
  s.student_id,
  s.name
ORDER BY rank_by_enrollments;
```

SQL Medium: Movies Watched by Viewers of 'Life of Pi'

📁 Database Description

The movie_streaming database is designed to manage and monitor digital content consumption across users. It maintains detailed records of user profiles, movie metadata, genre classifications, individual viewing history, and subscription plans. This system enables in-depth analysis of user

preferences, viewing trends, and billing activities, supporting both personalized recommendations and subscription management.

✔ Schema Specification

📄 Table: Users

Stores platform subscribers' information.

Column Name	Data Type	Constraints	Description
user_id	INT	PRIMARY KEY	Unique user identifier
name	VARCHAR(100)	NOT NULL	Full name of the user
email	VARCHAR(100)	UNIQUE NOT NULL	Email address
signup_date	DATE	NOT NULL	Date of registration

📄 Table: Genres

Categorizes movies into types like Drama, Action, etc.

Column Name	Data Type	Constraints	Description
genre_id	INT	PRIMARY KEY	Unique genre identifier
genre_name	VARCHAR(50)	NOT NULL	Name of the genre

📄 Table: Movies

Catalog of movies available on the platform.

Column Name	Data Type	Constraints	Description
movie_id	INT	PRIMARY KEY	Unique movie ID
title	VARCHAR(100)	NOT NULL	Movie title
release_year	INT		Year of release
genre_id	INT	FOREIGN KEY → Genres(genre_id)	Genre of the movie

 **Table: Watch_History**

Tracks which user watched what movie and when.

Column Name	Data Type	Constraints	Description
user_id	INT	FOREIGN KEY → Users(user_id)	Viewer
movie_id	INT	FOREIGN KEY → Movies(movie_id)	Movie watched
watch_date	DATE	NOT NULL	When it was watched
PRIMARY KEY	Composite	PRIMARY KEY (user_id, movie_id, watch_date)	Unique per watch event

 **Table: Subscriptions**

Tracks payment and plan details for users.

Column Name	Data Type	Constraints	Description
subscription_id	INT	PRIMARY KEY	Unique subscription entry
user_id	INT	FOREIGN KEY → Users(user_id)	Associated user
plan	VARCHAR(50)	NOT NULL	Subscription plan name
amount	DECIMAL(10,2)	NOT NULL	Amount billed
start_date	DATE	NOT NULL	Start of billing cycle
end_date	DATE	NOT NULL	End of billing cycle

✔ Test Data

Users

user_id	name	email	signup_date
801	Alice Moore	alice@stream.com	2023-01-15
802	Bob Green	bob@stream.com	2023-02-10
803	Carol White	carol@stream.com	2023-03-20
804	Dave Black	dave@stream.com	2023-04-05

Genres

genre_id	genre_name
1	Drama
2	Action
3	Comedy
4	Documentary

Movies

movie_id	title	release_year	genre_id
901	The Great Escape	1963	2
902	Life of Pi	2012	4
903	The Office (Series)	2005	3
904	Hamlet	1996	1

Watch_History

user_id	movie_id	watch_date
801	901	2023-08-01
801	902	2023-08-02
802	903	2023-08-03
802	904	2023-08-04
803	901	2023-08-05
803	902	2023-08-06
804	903	2023-08-07
804	904	2023-08-08
801	903	2023-08-09
802	902	2023-08-10
803	904	2023-08-11
804	901	2023-08-12

Subscriptions

subscription_id	user_id	plan	amount	start_date	end_date
1001	801	Premium	19.99	2023-08-01	2023-08-31
1002	802	Basic	9.99	2023-08-01	2023-08-31
1003	803	Premium	19.99	2023-08-01	2023-08-31
1004	804	Standard	14.99	2023-08-01	2023-08-31

Problem Statement

List all movies that have been watched by **any user who has watched 'Life of Pi'**. Duplicate movie titles must not appear in the result.

Query to be Implemented

Write a SQL query to retrieve:

- movie_id
- title

Only include movies watched by users who have also watched 'Life of Pi'.

📌 Expected Output

movie_id	title
901	The Great Escape
902	Life of Pi
903	The Office (Series)
904	Hamlet

 SQL

```
SELECT DISTINCT
  m.movie_id,
  m.title
FROM Watch_History wh
JOIN Watch_History wh2
  ON wh.user_id = wh2.user_id
JOIN Movies m
  ON wh.movie_id = m.movie_id
JOIN Movies m2
  ON wh2.movie_id = m2.movie_id
WHERE m2.title = 'Life of Pi';
```

movie_id	title
901	The Great Escape
902	Life of Pi
903	The Office (Series)
904	Hamlet

